◈ Queue Interface

66. What is Queue in Java?
67. How does Queue follow FIFO principle?
68. What are main methods of Queue?
69. When should you use Queue?
70. What is the difference between Queue and Deque?

◈ PriorityQueue

71. What is PriorityQueue?
72. How does PriorityQueue determine priority?
73. When should you use PriorityQueue?
74. Why does PriorityQueue not maintain insertion order?
75. How does natural ordering work in PriorityQueue?

◈ Map Interface

76. What is Map in Java?
77. How is Map different from Collection?
78. What are key methods of Map?
79. When should you use Map?
80. Why does Map store key-value pairs?

◈ HashMap

81. What is HashMap?
82. How does HashMap work internally?
83. When should you use HashMap?
84. Why is HashMap not synchronized?
85. What is collision in HashMap?

◈ LinkedHashMap

86. What is LinkedHashMap?
87. How is LinkedHashMap different from HashMap?
88. When should you use LinkedHashMap?
89. How does LinkedHashMap maintain order?
90. Why is LinkedHashMap slower than HashMap?

◈ TreeMap

91. What is TreeMap?
92. How does TreeMap sort keys?
93. When should you use TreeMap?
94. Why does TreeMap not allow null keys?
95. What is NavigableMap?

◈ Generics

96. What are Generics in Java?
97. Why are Generics used?
98. How do Generics improve type safety?
99. What is type erasure?
100. When should you use Generics?

◈ Wildcards & Bounds

101. What is wildcard in Generics?
102. What is upper bound wildcard (<? extends T>)?
103. What is lower bound wildcard (<? super T>)?
104. When should you use upper bound?
105. When should you use lower bound?
106. How does wildcard improve flexibility?

◈ Object Class

107. What is Object class in Java?
108. What are important methods of Object class?

109. How does equals() work?

110. What is hashCode() method?

111. Why override equals() and hashCode()?

112. What is toString() method?

113. What is clone() method?

114. What is finalize() method?

115. How does getClass() work?

MCQ Questions

---

1. Queue Output

```
Queue<Integer> q = new LinkedList<>();
q.add(10);
q.add(20);
q.add(30);
System.out.println(q.poll());
```

A. 30
B. 20
C. 10
D. Error

Answer: C
Explanation: Queue follows FIFO → first element (10) is removed.

---

2. Queue Peek

```
Queue<Integer> q = new LinkedList<>();
q.add(5);
q.add(10);
System.out.println(q.peek());
```

A. 5
B. 10
C. null
D. Error

Answer: A
Explanation: peek() returns head without removing.

---

3. PriorityQueue Order

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(30);
pq.add(10);
pq.add(20);
System.out.println(pq.poll());
```

A. 30
B. 10
C. 20
D. Error

Answer: B
Explanation: Natural ordering → smallest element first.

---

4. PriorityQueue Iteration

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(3);
pq.add(1);
pq.add(2);
System.out.println(pq);
```

A. [1,2,3]
B. [3,1,2]
C. Order not guaranteed
D. Error

Answer: C
Explanation: Internal heap structure, not sorted view.

---

5. HashMap Output

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "A");
map.put(2, "B");
map.put(1, "C");
System.out.println(map.get(1));
```

A. A
B. B
C. C
D. Error

Answer: C
Explanation: Key overwritten → latest value stored.

---

6. HashMap Null Key

```
Map<String, Integer> map = new HashMap<>();
map.put(null, 10);
map.put(null, 20);
System.out.println(map.get(null));
```

A. 10
B. 20
C. null
D. Error

Answer: B
Explanation: Only one null key allowed → overwritten.

---

7. LinkedHashMap Order

```
Map<Integer,String> map = new LinkedHashMap<>();
map.put(2,"B");
map.put(1,"A");
map.put(3,"C");
System.out.println(map);
```

A. {1=A,2=B,3=C}
B. {2=B,1=A,3=C}
C. Random order
D. Error

Answer: B
Explanation: Maintains insertion order.

---

8. TreeMap Sorting

```
Map<Integer,String> map = new TreeMap<>();
map.put(3,"C");
map.put(1,"A");
map.put(2,"B");
System.out.println(map);
```

A. {3=C,1=A,2=B}
B. {1=A,2=B,3=C}
C. Random
D. Error

Answer: B
Explanation: Sorted by keys.

---

9. TreeMap Null Key

```
TreeMap<Integer,String> map = new TreeMap<>();
map.put(null,"A");
```

A. null stored
B. Error
C. Works fine
D. Prints null

Answer: B
Explanation: TreeMap doesn't allow null keys.

---

10. HashMap Size

```
Map<Integer,Integer> map = new HashMap<>();
map.put(1,1);
map.put(1,2);
map.put(2,2);
System.out.println(map.size());
```

A. 3
B. 2
C. 1
D. Error

Answer: B

---

11. Generics Type Safety

```
List<String> list = new ArrayList<>();
list.add("A");
// list.add(10);
System.out.println(list.get(0));
```

A. A
B. 10
C. Error
D. Null

Answer: A
Explanation: Generics restrict type at compile time.

---

12. Raw Type

```
List list = new ArrayList();
list.add("A");
list.add(10);
System.out.println(list.get(1));
```

A. A
B. 10
C. Error
D. Null

Answer: B

---

13. Upper Bound

```
List<? extends Number> list = new ArrayList<Integer>();
// list.add(10);
System.out.println(list.size());
```

A. 1
B. 0
C. Error
D. Null

Answer: B
Explanation: Cannot add elements → only read.

---

14. Lower Bound

```
List<? super Integer> list = new ArrayList<Number>();
list.add(10);
System.out.println(list.get(0));
```

A. 10
B. Error
C. Object
D. Null

Answer: C
Explanation: Returns Object type.

---

15. equals() Method

```
String s1 = new String("A");
String s2 = new String("A");
System.out.println(s1.equals(s2));
```

A. true
B. false
C. Error
D. Null

Answer: A

---

16. hashCode()

```
String s1 = "A";
String s2 = "A";
System.out.println(s1.hashCode() == s2.hashCode());
```

A. true
B. false
C. Error
D. Random

Answer: A

---

17. toString()

```
Object obj = new Object();
System.out.println(obj.toString());
```

A. Object
B. ClassName@hashcode
C. Error
D. Null

Answer: B

---

18. Stack Behavior

```
Stack<Integer> st = new Stack<>();
st.push(10);
st.push(20);
System.out.println(st.pop());
```

A. 10
B. 20
C. Error
D. Null

Answer: B
Explanation: LIFO

---

19. Queue vs Deque

```
Deque<Integer> dq = new LinkedList<>();
dq.addFirst(10);
dq.addLast(20);
System.out.println(dq.removeFirst());
```

A. 20
B. 10

C. Error
D. Null

Answer: B

---

20. Map Iteration

```
Map<Integer,String> map = new HashMap<>();
map.put(1,"A");
map.put(2,"B");
for(Integer key : map.keySet()){
   System.out.print(map.get(key));
}
```

A. AB
B. BA
C. AB or BA
D. Error

Answer: C
Explanation: HashMap order not guaranteed.

Solve question by using Map

1. Two Sum

Problem:
Given an array of integers and a target, return indices of two numbers such that they add up to the target.

Example:
Input → nums = [2, 7, 11, 15], target = 9
Output → [0, 1]

---

2. First Non-Repeating Character

Problem:
Find the first character in a string that does not repeat.

Example:
Input → "leetcode"
Output → 'l'

---

3. Frequency of Elements

Problem:
Count frequency of each element in an array.

Example:
Input → [1, 2, 2, 3, 3, 3]
Output → {1=1, 2=2, 3=3}

---

4. Contains Duplicate

Problem:
Check if an array contains duplicate elements.

Example:
Input → [1, 2, 3, 1]
Output → true

---

5. Intersection of Two Arrays

Problem:
Find common elements between two arrays.

Example:
Input → [1,2,2,1], [2,2]
Output → [2]

□ Intermediate Level

---

6. Subarray Sum Equals K

Problem:
Find number of subarrays whose sum equals k.

Example:
Input → nums = [1,1,1], k = 2
Output → 2

---

7. Longest Substring Without Repeating Characters

Problem:
Find length of longest substring without repeating characters.

Example:
Input → "abcabcbb"
Output → 3

---

8. Group Anagrams

Problem:
Group words that are anagrams of each other.

Example:
Input → ["eat","tea","tan","ate","nat","bat"]
Output → [["eat","tea","ate"],["tan","nat"],["bat"]]

---

9. Top K Frequent Elements

Problem:
Find k most frequent elements in an array.

Example:
Input → nums = [1,1,1,2,2,3], k = 2
Output → [1,2]

---

10. Longest Consecutive Sequence

Problem:
Find the length of the longest consecutive sequence.

Example:
Input → [100,4,200,1,3,2]
Output → 4 (sequence → 1,2,3,4)